

Signal Breach - Projektdokumentation

Erstellen eines einfachen Computerspiels ohne Game Engine

Einzelarbeit im Themenbereich Game-Programmierung

Stand: 18. Juni 2026

PYTHON · PYGAME-CE · EIGENE GRAFIK · EIGENES AUDIO

1. Aufgabenstellung und Zielsetzung

Ziel der Arbeit war die eigenständige Programmierung eines funktionierenden und interaktiven Computerspiels ohne Game Engine. Vorgefertigte Spielmechaniken, fertige Grafikpakete und fremde Audio-Assets sollten nicht verwendet werden. Gleichzeitig sollte die Entwicklung auf den Kursübungen aufbauen und durch eine technische Dokumentation nachvollziehbar werden.

Signal Breach erfüllt diese Zielsetzung als vertikaler Arcade-Shooter. Das Spiel wird direkt in Python mit pygame-ce ausgeführt. pygame-ce stellt Fenster, Eingabe, einfache Zeichenoperationen und Audiowiedergabe bereit, übernimmt aber keine Spielregeln, Szenen, Physik, Gegnerlogik oder Levelprogression. Diese Bestandteile wurden im Projekt selbst implementiert.

Die Abgabe umfasst:

- ein direkt ausführbares Spiel,
- drei vollständige Sektoren mit Bosskämpfen,
- ein Punkte-, Combo- und Upgrade-System,
- persistente Statistiken und Einstellungen,
- automatisierte Tests,
- selbst erzeugte Grafik und Audioeffekte,
- eine Benutzer- und eine technische Dokumentation.

2. Bezug zu den Kursübungen

Die Übungen 003 und 004 bilden die fachliche Ausgangsbasis. Übung 003 führte eine Entity-Basisklasse, automatische Schüsse, Gegner, Hindernisse, Trefferpunkte und die Zustände Spielen und Game Over ein. Übung 004 ergänzte kachelbare Hintergründe, Punkte, Highscore, Upgrade-Shop, mehrere Level, besondere Gegnertypen und Bosskämpfe.

Das Abschlussprojekt übernimmt diese Lernziele, entwickelt sie aber zu einem eigenständigen System weiter:

- Statt einer ausschließlich mausgesteuerten Bewegung gibt es Tastatur- und Maussteuerung in zwei Dimensionen.
- Zustände werden durch einen zentralen Zustandsautomaten verwaltet.
- Level bestehen aus unveränderlichen Definitionen und getrennten Laufzeitdaten.
- Gegner besitzen klar getrennte Bewegungs- und Angriffsmuster.
- Bosse wechseln abhängig von ihren Lebenspunkten zwischen mehreren Phasen.
- Der Hintergrund besteht nicht aus Bilddateien, sondern wird vollständig prozedural gezeichnet.
- Audioeffekte werden als Wellenformen berechnet und nicht geladen.
- Persistenz verwendet validiertes JSON und atomisches Speichern.
- Die neue Logik wird durch eigenständige Unit- und Integrationstests abgesichert.

Die Übungsordner bleiben unverändert als Entwicklungsgeschichte im Repository erhalten. Signal Breach verwendet ihre Bilder und Leveldateien nicht.

3. Spielkonzept

3.1 Thema

Der Spieler steuert ein Reparaturprogramm durch ein beschädigtes Datennetz. Gegner sind fehlerhafte Prozesse, Projektile sind Datenimpulse und die Bosse stellen Sektorwachen dar. Der abstrakte technische Rahmen ermöglicht einen eigenständigen Stil aus Neonfarben, Rasterlinien und geometrischen Formen.

3.2 Kernschleife

Eine Spielrunde folgt dieser Schleife:

1. Im Menü wird eine neue Verbindung gestartet.
2. In einem Sektor erscheinen zeitgesteuerte Gegnerwellen.
3. Der Spieler weicht aus und zerstört Gegner durch automatisches Feuer.
4. Nach der letzten Welle erscheint eine Sektorwache.
5. Nach dem Bosskampf öffnet sich ein Upgrade-Shop.
6. Nach dem dritten Boss endet die Runde mit einem Sieg.
7. Bei null Lebenspunkten endet die Runde sofort mit einer Niederlage.

Die kurze Schleife aus Bewegen, Ausweichen, Abschießen und Verbessern soll direkt verständlich sein. Steigende Gegnerdichte und neue Muster erzeugen Progression ohne zusätzliche komplizierte Systeme.

3.3 Steuerung

WASD und Pfeiltasten bewegen das Schiff frei innerhalb der Spielfläche. Eine Mausbewegung setzt die Position alternativ direkt. Das Schiff feuert automatisch nach oben. Dadurch liegt der Schwerpunkt auf Positionierung und Ausweichen. Die Leertaste aktiviert einen Impuls, der nahe gegnerische Projektile zerstört. Seine Abklingzeit verhindert dauerhaftes Spammen.

P oder Escape pausieren die Simulation. F schaltet Vollbild um, M schaltet Audio ein oder aus. Im Shop kaufen die Zifferntasten 1 bis 4 Verbesserungen. Enter startet und bestätigt Zustandswechsel.

4. Funktionale Anforderungen

4.1 Zustände

Das Spiel besitzt sechs explizite Zustände:

- **menu**: Titel, Ziel, Steuerung und persistente Bestwerte.
- **playing**: aktive Simulation und Kampflogik.
- **paused**: unveränderte Spielwelt mit Pausenüberlagerung.
- **shop**: Kauf von Verbesserungen zwischen Sektoren.
- **game_over**: Statistik nach dem Tod des Spielers.
- **won**: Statistik nach Abschluss aller drei Sektoren.

Nur im Zustand **playing** wird die Simulation aktualisiert. Dadurch können Pause und Menüs keine versteckten Gegnerbewegungen oder Timer verursachen.

4.2 Kampfsystem

Der Spieler besitzt Lebenspunkte, eine kurze Unverwundbarkeitszeit nach Treffern, automatische Schüsse und den Impuls. Projektilschaden, Feuerrate, Geschwindigkeit und Reichweite stammen aus den aktuellen Spielerwerten. Kollisionen werden über achsenparallele Rechtecke geprüft.

Die Kollisionsreihenfolge ist fest definiert:

1. Spielerprojekteile gegen lebende Gegner.
2. Gegnerprojekteile gegen den Spieler.
3. Gegnerkörper gegen den Spieler.
4. Entfernen zerstörter Objekte.
5. Zustandswechsel bei Tod oder beendetem Bosskampf.

Ein Projektil trifft höchstens einen Gegner. Normale Gegner können Fragmente erzeugen. Bosse geben höhere Punkt- und Währungswerte.

4.3 Gegner

Fünf normale Gegnervarianten werden aus einer gemeinsamen Klasse konfiguriert:

- Drifter bewegen sich gerade nach unten und schießen gezielt.
- Weaver bewegen sich in einer Sinuskurve und feuern langsamer.
- Charger erfassen nach einer Vorbereitungszeit die Spielerposition und beschleunigen dorthin.
- Splitter besitzen mehr Lebenspunkte und erzeugen beim Tod zwei Fragmente.
- Fragmente sind klein, schnell und bewegen sich diagonal.

Diese Varianten kombinieren dieselben Grundbausteine unterschiedlich. Dadurch bleibt die Implementierung überschaubar, während sich die Spielsituationen deutlich unterscheiden.

4.4 Bosse

Jeder Sektor besitzt genau eine Sektorwache. Die Bossklasse erweitert die normale Gegnerklasse um maximale Lebenspunkte, Phasen, Angriffstimer, Warnfeedback und ein Angriffsmuster.

Der erste Boss verwendet gerichtete Fächerschüsse. Der zweite Boss kombiniert Fächer mit beschworenen Weaver-Gegnern. Der dritte Boss erzeugt in späteren Phasen radiale Projektile. Ab zwei Dritteln und einem Drittel der Lebenspunkte steigt die Phase. Angriffstempo und Projektilzahl nehmen zu.

4.5 Punkte und Combo

Ein Abschuss vergibt Basispunkte und Datenfragmente. Erfolgt der nächste Abschuss innerhalb des Combo-Fensters, erhöht sich die Combo. Jede zusätzliche Stufe gibt 50 Bonuspunkte. Läuft das Zeitfenster ab oder wird der Spieler getroffen, fällt die Combo auf null zurück. Der höchste Wert eines Durchlaufs wird separat gespeichert.

4.6 Upgrades

Zwischen Sektoren stehen vier Verbesserungen zur Verfügung:

- Schaden erhöht den Projektilschaden um eins.
- Feuerrate reduziert die Wartezeit zwischen Schüssen um zwölf Prozent.
- Integrität erhöht maximale Lebenspunkte um 25 und heilt vollständig.
- Projektiltempo erhöht Geschwindigkeit und Reichweite.

Jede Verbesserung besitzt vier Stufen. Die Kosten entsprechen den Basiskosten multipliziert mit der nächsten Stufe. Ein Kauf ohne ausreichende Währung oder oberhalb der Maximalstufe verändert keine Daten.

5. Leveldesign und Progression

Sektoren werden in `levels.py` als unveränderliche Datensätze beschrieben. Eine Welle enthält Zeitpunkt, Gegnertyp, Anzahl und Formation. Die Laufzeitklasse speichert nur verstrichene Zeit, den Index der nächsten Welle und den Bossstatus.

Sektor 1 führt Drifter und Weaver ein. Sektor 2 ergänzt Charger und Splitter sowie dichtere Wellen. Sektor 3 kombiniert alle Gegnertypen mit höheren Geschwindigkeiten, Lebenspunkten und Schäden. Die Laufzeiten betragen ungefähr 42, 50 und 58 Sekunden vor dem jeweiligen Boss.

Formationen verteilen Gegner als Linie, versetzte Reihe oder V-Formation im sicheren horizontalen Bereich. Eine Welle wird exakt einmal ausgelöst. Der Boss darf erst erscheinen, wenn alle Wellenzeitpunkte erreicht und alle normalen Gegner entfernt wurden.

Die Progression entsteht nicht nur durch höhere Zahlen. Sektor 1 vermittelt zunächst die Grundbewegung und die Bedeutung gezielter Schüsse. Sektor 2 verlangt häufigere Richtungswechsel durch Charger und zusätzliche Ziele nach einem Splitter-Abschuss. Sektor 3 kombiniert diese Anforderungen mit dichterem Projektilfeuer und einem radial angreifenden Boss.

6. Softwarearchitektur

6.1 Modulübersicht

`main.py` initialisiert pygame, erzeugt ein virtuelles 960-mal-720-Pixel-Canvas und skaliert es unter Beibehaltung des Seitenverhältnisses auf die aktuelle Fenstergröße. Mauskoordinaten werden zurück auf das virtuelle Canvas abgebildet.

`signal_breach/model.py` enthält die Zustandsaufzählung sowie reine Datenklassen für Profil, Spielerwerte und Durchlauf. Dieses Modul hängt nicht von pygame ab.

`signal_breach/persistence.py` lädt und speichert das Profil. Ungültige Daten werden bereinigt. Schreiben erfolgt zuerst in eine temporäre Datei, die anschließend atomisch die Zielfeile ersetzt.

`signal_breach/scoring.py` enthält Punkte-, Combo- und Upgrade-Regeln. Diese Regeln arbeiten ausschließlich auf den Daten aus `model.py`.

`signal_breach/entities.py` enthält Spieler, Projektile, Gegner und Bosse. Die Klassen aktualisieren ihre Simulation, zeichnen sich aber nicht selbst.

`signal_breach/levels.py` enthält Sektordefinitionen und die zeitgesteuerte Erzeugung von Wellen.

`signal_breach/game.py` verbindet Eingabe, Zustände, Entities, Level, Kollision, Punkte, Audio und Persistenz. Die Klasse `Game` bildet die zentrale Anwendungsschnittstelle.

`signal_breach/rendering.py` zeichnet alle Zustände und Objekte. Das Rendering liest Spieldaten, verändert aber keine Regeln.

`signal_breach/audio.py` erzeugt kurze Mono-Wellenformen und kapselt Audiowiedergabe. Falls kein Audiogerät verfügbar ist, bleibt das Spiel ohne Fehlermeldung spielbar.

6.2 Abhängigkeitsrichtung

Die Abhängigkeiten verlaufen von außen nach innen:

```
main.py
  -> Game + Renderer
Game
```

```
-> Levels + Entities + Scoring + Persistence + Audio
Levels
-> Entities
Scoring + Persistence
-> Model
Model
-> Python-Standardbibliothek
```

Rendering kennt den aktuellen Zustand, aber die Spiellogik kennt keine Schriftarten oder Zeichenoperationen. Persistenz kennt keine pygame-Klassen. Diese Trennung verbessert Testbarkeit und verhindert zyklische Abhängigkeiten.

6.3 Ablauf eines Frames

Im Hauptprogramm wird pro Frame folgende Reihenfolge verwendet:

1. pygame-Ereignisse auslesen und an `Game.handle_event` übergeben.
2. Vollbildwunsch verarbeiten.
3. aktuell gehaltene Bewegungstasten in einen Richtungsvektor umwandeln.
4. Delta-Zeit durch die Clock bestimmen.
5. `Game.update` aufrufen.
6. den Zustand auf das virtuelle Canvas zeichnen.
7. Canvas proportional in das Fenster skalieren.
8. Bildschirmpuffer austauschen.

Die Delta-Zeit wird in der Simulation auf höchstens ein Dreißigstel Sekunde begrenzt. Ein langsamer Frame kann dadurch keine extrem großen Bewegungs- oder Kollisionssprünge erzeugen.

7. Zentrale Algorithmen

7.1 Bewegung und Begrenzung

Gleichzeitige Richtungen werden als Vektor zusammengefasst. Diagonale Vektoren werden normalisiert, damit diagonale Bewegung nicht schneller ist. Danach wird Geschwindigkeit mal Delta-Zeit addiert. Die Koordinaten werden auf den sichtbaren Bereich unterhalb des HUD begrenzt.

7.2 Projektilreichweite

Jedes Projektil summiert die Länge seines Bewegungsvektors. Sobald die konfigurierte Maximaldistanz erreicht ist oder das Projektil einen Rand mit Sicherheitsabstand verlässt, wird es als nicht lebend markiert. Die zentrale Bereinigung entfernt es nach der Kollisionsphase.

7.3 Sinusbewegung

Weaver speichern ihre ursprüngliche X-Position. Die horizontale Position wird aus dem Sinus ihres Alters berechnet, während Y linear steigt. Dadurch ist die Bewegung unabhängig von der Bildrate und bleibt reproduzierbar.

7.4 Bossphasen

Das Verhältnis aus aktuellen und maximalen Lebenspunkten bestimmt die Phase. Oberhalb von 66 Prozent gilt Phase eins, oberhalb von 33 Prozent Phase zwei, darunter Phase drei. Die Phase reduziert das Angriffsintervall und erhöht bei Fächer- und Radialangriffen die Projektilanzahl.

7.5 Persistenz

Beim Laden muss der JSON-Inhalt ein Objekt sein. Zahlen werden sicher konvertiert und auf sinnvolle Bereiche begrenzt. Fehlt die Datei oder ist sie beschädigt, wird ein Standardprofil verwendet. Highscore und beste Combo können nie negativ werden; die freigeschaltete Sektornummer liegt zwischen eins und drei.

Beim Speichern wird formatierter JSON-Text in eine temporäre Nachbardatei geschrieben. Erst danach ersetzt diese Datei das Profil. Ein Programmabbruch während des Schreibens beschädigt dadurch nicht die zuletzt gültige Datei.

7.6 Kollisionen

Alle interaktiven Objekte stellen ein Rechteck um ihre sichtbare Form bereit. Pro Spielerprojektil wird der erste lebende, überlappende Gegner gesucht. Nach einem Treffer wird das Projektil deaktiviert. Stirbt das Ziel, löst die Spiellogik Punkte, Währung, Audiofeedback und mögliche Splitter-Fragmente aus.

Gegnerprojekteile werden separat geprüft, damit der Impuls ausschließlich feindliche Schüsse beeinflusst. Die Unverwundbarkeitszeit des Spielers verhindert, dass mehrere überlappende Objekte im selben Moment die gesamten Lebenspunkte entfernen.

8. Gestaltung ohne vorgefertigte Assets

Das Abschlussprojekt lädt keine Bilddateien. Spieler, Gegner und Bosse bestehen aus Polygonen, Linien, Rechtecken und Kreisen. Transparente Zusatzflächen erzeugen Leuchteffekte. Hintergrundverlauf, perspektivisches Raster, Sterne, HUD, Shopkarten und Überlagerungen werden in jedem Frame durch pygame-Zeichenfunktionen erzeugt.

Der Stil verwendet einen tiefblauen Hintergrund und helle Akzentfarben. Spielerprojekteile sind cyan, gegnerische Projekteile rot. Gegnertypen besitzen eigene Farben und Silhouetten. Treffer blinken weiß, Bosse besitzen eine gut sichtbare Lebensleiste und Phase.

Auch die Audioeffekte sind Eigenleistung. Die Funktion `tone` berechnet für jedes Sample eine Sinusschwingung. Eine Hüllkurve reduziert die Lautstärke zum Ende, ein Frequenz-Sweep erzeugt auf- oder absteigende Effekte. Daraus entstehen Schuss-, Treffer-, Explosion-, Kauf-, Warn- und Impulssounds. Es werden keine WAV- oder Musikdateien geladen.

9. Fehlerbehandlung

Das Profil ist optional. Fehlende, unlesbare oder syntaktisch ungültige Dateien führen zu sicheren Standardwerten. Unbekannte Gegner-, Boss- oder Upgrade-Namen lösen früh eine aussagekräftige Ausnahme aus, weil sie Programmierfehler in den Leveldaten darstellen.

Audio ist nicht spielentscheidend. Schlägt die Initialisierung des Mixers fehl, arbeitet `AudioManager` als stummer Adapter weiter. Die Simulation und Darstellung bleiben vollständig verfügbar.

Fenstergrößen werden unabhängig vom Seitenverhältnis akzeptiert. Nicht verwendete Bereiche werden schwarz gefüllt. Dadurch werden Spielgrafik und Eingabekoordinaten nicht verzerrt.

10. Tests und Qualitätssicherung

Die Tests verwenden die Python-Standardbibliothek `unittest`. Für automatisierte Läufe werden SDL-Video und SDL-Audio auf den Dummy-Treiber gesetzt. Dadurch ist kein sichtbares Fenster und kein Audiogerät erforderlich.

10.1 Modell und Persistenz

Geprüft werden fehlende und ungültige Profildateien, Rundreisen durch Speichern und Laden, Wertebegrenzung und Standardwerte eines neuen Durchlaufs.

10.2 Punkte und Shop

Tests prüfen Combo-Aufbau, Ablauf des Zeitfensters, Zurücksetzen bei Treffern, Währungsvergabe, Kostensteigerung, Maximalstufen und abgelehnte Käufe ohne Datenänderung.

10.3 Entities

Geprüft werden Spielfeldgrenzen, Unverwundbarkeit, automatische Schüsse mit aktuellen Werten, Impulsradius, Projektilreichweite, unterschiedliche Bewegungsprofile, Splitter-Fragmente und Bossphasen.

10.4 Level und Zustände

Die Tests verifizieren drei aufsteigende Sektoren, unterschiedliche Bossmuster, einmalige Wellenauslösung, sichere Formationspositionen und die korrekte Bossfreigabe. Integrationstests durchlaufen Menü, Pause, Shop, Sieg, Neustart und Game Over.

10.5 Rendering

Jeder Zustand wird auf dieselbe 960-mal-720-Fläche gerendert. Der Test bestätigt, dass dabei tatsächlich mehrere Farben und damit sichtbare Inhalte entstehen. Zusätzlich werden für die Abnahme Screenshots der zentralen Zustände erzeugt und visuell geprüft.

10.6 Regression

Die Tests der Übungen 003 und 004 bleiben Bestandteil der Gesamtsuite. Damit zeigt die Abgabe sowohl den funktionierenden Kursstand als auch das neue Abschlussprojekt. Zum Abnahmezeitpunkt bestanden alle 66 automatisierten Tests.

11. Bedienbarkeit und Feedback

Das Startmenü erklärt alle wesentlichen Eingaben, ohne den Spielbildschirm zu überladen. Im Spiel zeigt das HUD Lebenspunkte, Sektorfortschritt, Punktzahl, Datenfragmente, Combo und Impulsaufladung. Während eines Bosskampfes kommen Bosslebenspunkte und Phase hinzu.

Treffer, Zerstörung, Impuls und Käufe besitzen jeweils visuelles und akustisches Feedback. Ein roter Bildschirmblitz signalisiert erlittenen Schaden. Statusmeldungen kündigen Sektoren und Bosse an. Der Shop markiert bezahlbare Optionen farbig und zeigt Kosten, Stufe und Maximalwert.

Die feste interne Auflösung sorgt dafür, dass HUD und Kollisionsraum unabhängig von Fenstergröße oder Vollbild identisch bleiben. Das Seitenverhältnis wird beibehalten. Mauskoordinaten werden aus dem tatsächlichen Fenster auf die interne Auflösung zurückgerechnet.

12. Eigenleistung

Für Signal Breach wurden folgende Bestandteile eigenständig erstellt:

- Spielidee, Thema und visuelles Konzept,
- Zustandsautomat und Hauptschleife,
- Bewegungs-, Schuss-, Kollisions- und Schadenssystem,
- fünf Gegnerprofile und drei Bossmuster,
- alle Sektor- und Wellendaten,

- Punkte-, Combo-, Währungs- und Upgrade-System,
- JSON-Persistenz,
- sämtliche prozeduralen Grafiken,
- mathematische Audioerzeugung,
- automatisierte Tests,
- Benutzer- und Projektdokumentation.

Es wurden keine fertigen Spielmechaniken und keine fremden Medien in das Abschlussprojekt eingebunden. pygame-ce wird als Bibliothek für den technischen Zugriff auf SDL verwendet und ist keine Game Engine.

13. Grenzen und mögliche Erweiterungen

Der Umfang wurde bewusst auf eine vollständige, gut testbare Einzelspieler-Partie begrenzt. Es gibt keine Netzwerkfunktion, keinen externen Leveleditor und keine frei konfigurierbare Tastenbelegung. Die prozeduralen Sounds sind kurz und funktional, ersetzen aber keine komponierte Musik.

Sinnvolle spätere Erweiterungen wären Controller-Unterstützung, zusätzliche barrierefreie Farbpaletten, ein in das Spiel integriertes Tutorial, weitere Wellenformationen und eine Replay-Funktion auf Basis aufgezeichneter Eingaben.

14. Fazit

Signal Breach entwickelt die Mechaniken der Kursübungen zu einem geschlossenen Computerspiel weiter. Die Abgabe enthält einen vollständigen Ablauf vom Menü über drei zunehmend schwierige Sektoren und Upgrade-Phasen bis zu Sieg oder Niederlage. Durch die Trennung von Regeln, Laufzeit, Rendering, Audio und Persistenz bleibt der Quellcode nachvollziehbar. Automatisierte Tests sichern die zentralen Regeln und Zustandswechsel ab.

Die vollständig prozedurale Darstellung und Audioerzeugung belegen, dass das Projekt ohne vorgefertigte Assets auskommt. Gleichzeitig bleibt das Spiel direkt bedienbar und liefert klares Feedback zu allen wichtigen Aktionen.

15. Quellen und Werkzeuge

- Python Software Foundation: Python 3 Dokumentation, <https://docs.python.org/3/>
- pygame-ce Community: pygame-ce Dokumentation, <https://pyga.me/docs/>
- Astral: uv Dokumentation, <https://docs.astral.sh/uv/>
- ReportLab: ReportLab PDF Toolkit Dokumentation, <https://docs.reportlab.com/>

Die Quellen wurden als technische Referenz für Sprache und Bibliotheks-APIs verwendet. Es wurden keine externen Spielassets, Level, Gegnerlogiken oder fertigen Mechaniken übernommen.